Лабораторна робота №3

Віддалене налаштування мережі з використанням SSH-over-SCTP

Мета роботи — розібрати протокол SCTP, його переваги у порівнянні з TCP; навчитися встановлювати SSH з’єднання.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Що таке SCTP?

Протокол передачі потокового керування (SCTP) - це протокол рівня транспорту, розроблений для надання надійного, орієнтованого на повідомлення зв'язку між двома кінцевими точками в комп'ютерній мережі. Він був розроблений як альтернатива протоколу передачі контролю TCP (Transmission Control Protocol) та протоколу дейтаграм (UDP), щоб вирішити певні обмеження та вимоги, які ці протоколи не могли повністю задовольнити.

Основні особливості SCTP:

- **Орієнтований на повідомлення зв'язок**: На відміну від TCP, який надає абстракцію байтового потоку, SCTP орієнтований на повідомлення. Це означає, що дані передаються у вигляді окремих повідомлень, а не безперервного потоку байтів. Кожне повідомлення зберігає свої межі, що полегшує розділення окремих частин даних додатками.

- **Багатоадресний зв'язок (Multi-Homing)**: SCTP підтримує концепцію декількох IP-адрес, пов'язаних з однією кінцевою точкою. Це дозволяє забезпечити більшу надійність у випадках, коли мережеві шляхи можуть бути ненадійними або зайвими. Якщо один мережевий шлях виходить з ладу, SCTP може швидко переключитися на інший.

- **Підтримка багатьох потоків (Multi-Stream)**: SCTP дозволяє встановлювати кілька логічних потоків даних в межах однієї SCTP-асоціації. Кожен потік може використовуватися для різних цілей, що дозволяє ефективно мультиплексувати дані для різних потреб програм.

- **Впорядкована та неупорядкована доставка**: SCTP надає можливість доставки повідомлень як впорядковано, так і неупорядковано. Впорядкована доставка забезпечує приймання повідомлень у тому ж порядку, в якому вони були відправлені, тоді як неупорядкована доставка дозволяє повідомленням надходити в будь-якому порядку, але в межах певного потоку.

- **Виявлення та коригування помилок**: SCTP містить вбудовані механізми для виявлення та коригування помилок, що підвищує його надійність в мережах з різним рівнем надійності та якості.

- **Контроль перенасичення**: SCTP реалізовує механізми контролю перенасичення для запобігання перенасиченню мережі та забезпечення справедливого використання мережевих ресурсів між різними з'єднаннями.

- **Тримання з'єднання**: SCTP використовує повідомлення "пінгування" для відстеження стану з'єднань та кінцевих точок. Механізми тримання з'єднання допомагають зберігати зв’язок навіть при наявності збоїв в мережі.

Не дивлячись на те, що даний протокол має багато сильних сторін, він не є поширеним через:

- **Введення і новітність**: SCTP був введений пізніше, ніж вже встановлені та широко використовувані протоколи, такі як TCP та UDP. Це означає, що багато додатків і пристроїв вже були налаштовані на використання TCP та UDP, і перехід на SCTP вимагає змін в їхньому програмному та апаратному забезпеченні.

- **Підтримка в ОС та мережевих пристроях**: Багато операційних систем та

мережевих пристроїв підтримують TCP та UDP як основні протоколи. Підтримка SCTP може бути обмеженою або взагалі відсутньою в деяких пристроях та операційних системах, що ускладнює його впровадження.

- **Відсутність широкої потреби**: Більшість додатків можуть працювати ефективно з TCP та UDP. Тільки деякі специфічні сценарії, такі як додатки реального часу, вимагають переваг, які надає SCTP, такі як більш надійна доставка повідомлень.

- **Складність використання**: В порівнянні з TCP та UDP, SCTP може виглядати більш складним для розробників. Відсутність досвіду з цим протоколом та додаткові можливості, такі як підтримка багатьох потоків, можуть призвести до недорозумінь та помилок у розробці.

- **Сегментація ринку**: Через обмежену популярність SCTP, розробники можуть не бачити значної користі у підтримці цього протоколу, особливо якщо їхня аудиторія користується переважно TCP та UDP.

Алгоритм роботи SCTP:

1. **Асоціація (Association)**: SCTP встановлює логічне з'єднання між двома кінцевими точками, яке називається асоціацією. Кожна асоціація визначає потік даних, який може бути розділений на низку потоків. Це дозволяє SCTP передавати дані паралельно між різними потоками.

2. **Потоки (Streams)**: Кожен потік у SCTP веде свою власну послідовність передачі даних. Це означає, що дані з різних потоків можуть бути оброблені незалежно один від одного. Кожен потік використовує унікальний номер для ідентифікації послідовності передачі даних.

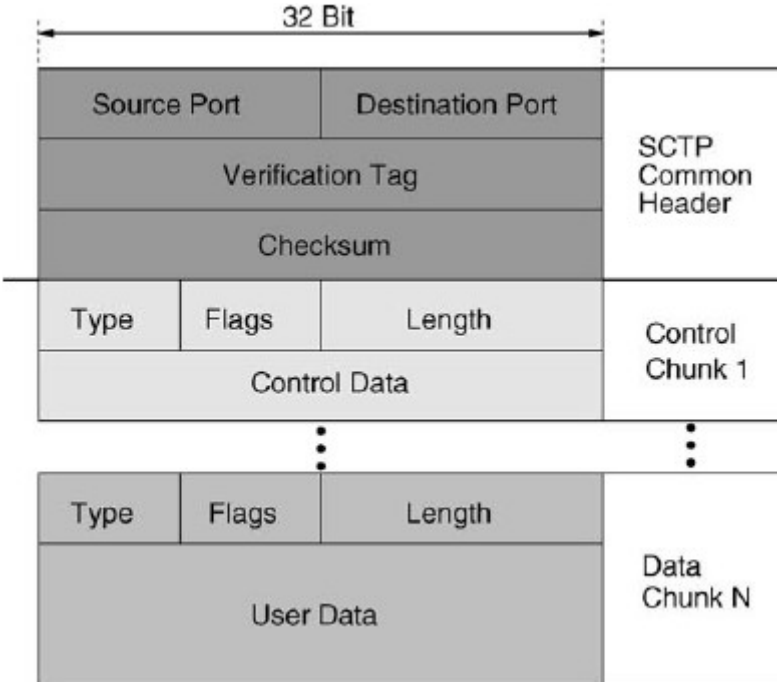
3. **Заголовок SCTP (SCTP Header)**: Кожен пакет SCTP має свій заголовок, який містить інформацію про асоціацію, потік, послідовність та інші керуючі дані. Ця інформація дозволяє отримувачеві відновити правильну послідовність даних та здійснити відповідні дії з обробки.

4. **Керування Передачею (Transmission Control)**: SCTP здійснює контроль за розміром вікна передачі для кожного потоку. Це дозволяє регулювати швидкість передачі даних відправником, що зменшує можливість перевантаження мережі або отримувача.

5. **Вікно передачі (Windowing)**: SCTP використовує вікно передачі для управління потоком даних. Вікно вказує максимальну кількість байтів, які можуть бути передані відправником, не чекаючи підтвердження від отримувача.

6. **Контроль З'єднання (Connection Control)**: SCTP має механізми виявлення та відновлення з'єднання в разі втрати пакетів або недоставки даних. Це забезпечує надійну передачу даних у мережі.

7. **Аутентифікація та Захист (Authentication and Security)**: SCTP підтримує механізми аутентифікації та шифрування для забезпечення безпеки передачі даних.



Що таке SSH?

SSH (Secure Shell) - це криптографічний мережевий протокол, призначений для безпечного з'єднання та обміну даними між комп'ютерами через незахищені мережі, такі як Інтернет. Використовуючи SSH, користувачі можуть входити на віддалені комп'ютери та виконувати команди, передавати файли та здійснювати інші мережеві операції з високим рівнем захисту. SSH забезпечує шифрування даних та аутентифікацію, що робить його важливим інструментом для забезпечення приватності та безпеки під час роботи з віддаленими системами.

ПІДГОТОВКА ДО ВИКОНАННЯ ЛАБОРАТОРНОЇ РОБОТИ №3

Перед виконанням лабораторної роботи рекомендується ознайомитись з відповідним розділом лекційного матеріалу курсу та засвоїти теоретичний матеріал.

ВИМОГИ ДО ОФОРМЛЕННЯ ЗВІТУ

Звіт з лабораторної роботи обов’язково повинен містити наступну інформацію:
- назва комп’ютерного практикуму;
- мета роботи;
- постановка завдання і алгоритм його розв’язання;
- відповіді на завдання у текстовому форматі та графічними зображеннями за необхідності.

Завдання на лабораторну роботу

Завдання 3.1. Налаштувати середовище

3.1.1 Dockefile

```
└─$ cat Dockerfile
FROM ubuntu:20.04
ARG DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y vim iproute2 iputils-ping \
                                         nmap net-tools ethtool ifupdown \
                                         netcat iptables tcpdump bridge-utils \
                                         arping

RUN apt-get install -y python3-pip
RUN pip3 install pysctp==0.7.2

RUN apt-get install -y socat openssh-server
RUN echo 'root:12345' | chpasswd
#RUN systemctl start ssh
ENTRYPOINT ["/usr/sbin/init"]
```

3.1.2. docker-compose.yml

```
└─$ cat docker-compose.yml
services:
  host1:
    build:
      context: .
      dockerfile: Dockerfile
    image: mutap_ubuntu
    hostname: host1
    container_name: host1
    tty: true
    networks:
      default:
        ipv4_address: 10.10.0.2
    privileged: true
  host2:
    build:
      context: .
      dockerfile: Dockerfile
    image: mutap_ubuntu
    hostname: host2
    container_name: host2
    tty: true
    networks:
      default:
        ipv4_address: 10.10.0.3
    privileged: true

networks:
  default:
    driver: bridge
    ipam:
      config:
        - subnet: 10.10.0.0/24
```

Завдання 3.2. Отримати дамп з SCTP пакетами

3.2.1. Створити скрипт server.py на host1

```
root@host1:/tmp# cat server.py
#!/usr/bin/python3.8
import socket
import sctp

server = sctp.sctpsocket_tcp(socket.AF_INET)
server.bind(('0.0.0.0', 12345))
server.listen(1)

print("Server listening...")
connection, address = server.accept()
print(f"Connected by: {address}")

while True:
    data = connection.recv(1024)
    if not data:
        break
    print(f"Received: {data.decode('utf-8')}")
    connection.send(data)  # Echo the received data

connection.close()
```

3.2.2. Створити скрипт client.py на host2

```
root@host2:/tmp# cat client.py
#!/usr/bin/python3.8
import socket
import sctp

client = sctp.sctpsocket_tcp(socket.AF_INET)
client.connect(('10.10.0.2', 12345))

message = "Hello, server!"
client.send(message.encode('utf-8'))

response = client.recv(1024)
print(f"Server Response: {response.decode('utf-8')}")

client.close()
```

3.2.3. Запустити Wireshark на прослуховування бриджа

3.2.4. Запустити server.py

3.2.5. Запустити client.py

3.2.6. Описати процедуру передачі даних з використанням отриманого дампу та зазначенням основних повідомлень

Завдання 3.3. Налаштувати SSH-over-SCTP

3.3.1. Дозволити root login

```
root@host1:/# cat /etc/ssh/sshd_config | grep PermitRootLogin
#PermitRootLogin prohibit-password
# the setting of "PermitRootLogin without-password".
PermitRootLogin yes
```

3.3.2. Перезапустити sshd

```
root@host1:/# cat /etc/ssh/sshd_config | grep PermitRootLogin
#PermitRootLogin prohibit-password
# the setting of "PermitRootLogin without-password".
PermitRootLogin yes
root@host1:/# systemctl restart sshd; systemctl status sshd
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: e
     Active: active (running) since Sat 2023-08-12 09:26:42 UTC; 3ms ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 201 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 202 (sshd)
      Tasks: 1 (limit: 5716)
     Memory: 1.2M
        CPU: 21ms
     CGroup: /system.slice/ssh.service
             └─202 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups

Aug 12 09:26:42 host1 systemd[1]: Starting OpenBSD Secure Shell server...
Aug 12 09:26:42 host1 sshd[202]: Server listening on 0.0.0.0 port 22.
Aug 12 09:26:42 host1 sshd[202]: Server listening on :: port 22.
Aug 12 09:26:42 host1 systemd[1]: Started OpenBSD Secure Shell server.
```

3.3.3. Спробувати підключитись по SSH до host1

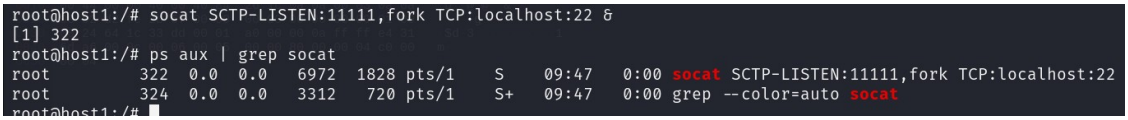
```
root@host2:/# ssh root@10.10.0.2
root@10.10.0.2's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.18.0-kali5-amd64 x8

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

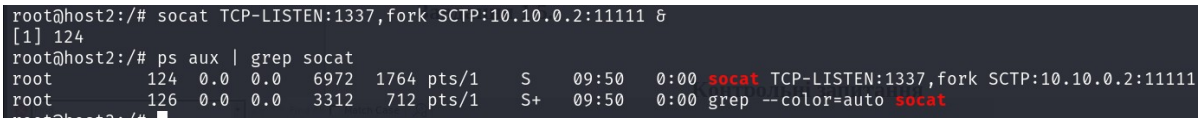
This system has been minimized by removing packages and conten
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Aug 12 09:23:36 2023 from 10.10.0.3
```

3.3.4. Прокладати тунель починаємо із запуску прослуховування порту 11111/SCTP та перенаправляння будь-якого трафіку на порт 22/TCP на host1.

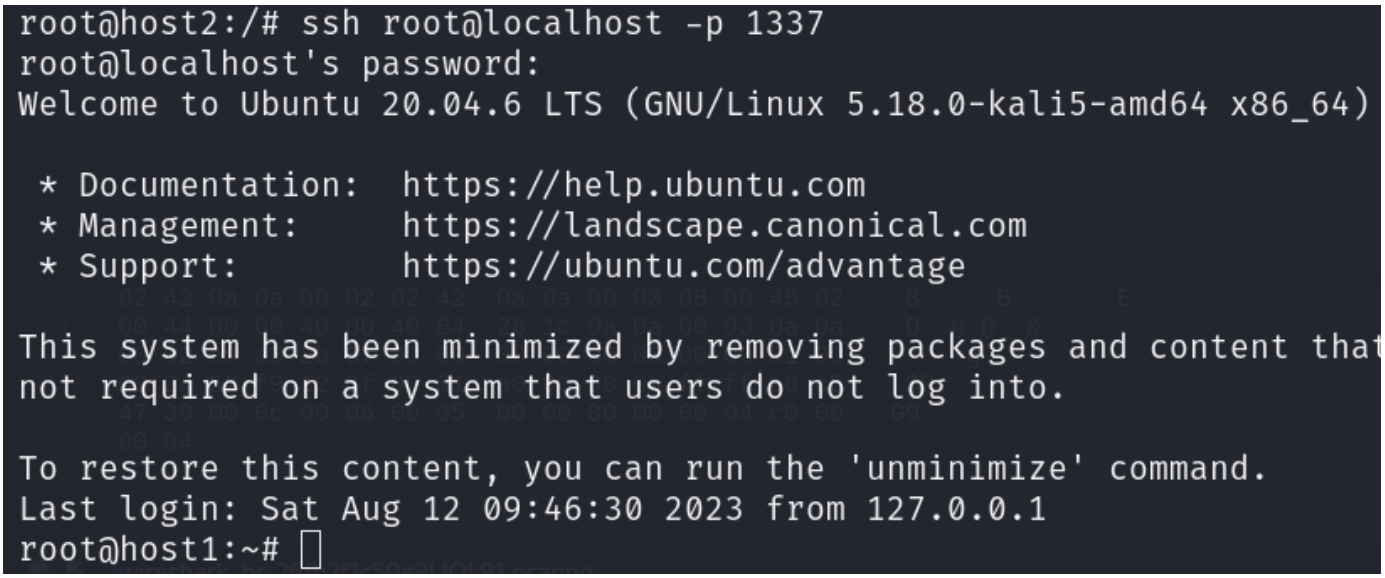


3.3.5. На host2 також знадобиться socat, що працює з підтримкою SCTP. Робимо схожий порт форфардінг на host2

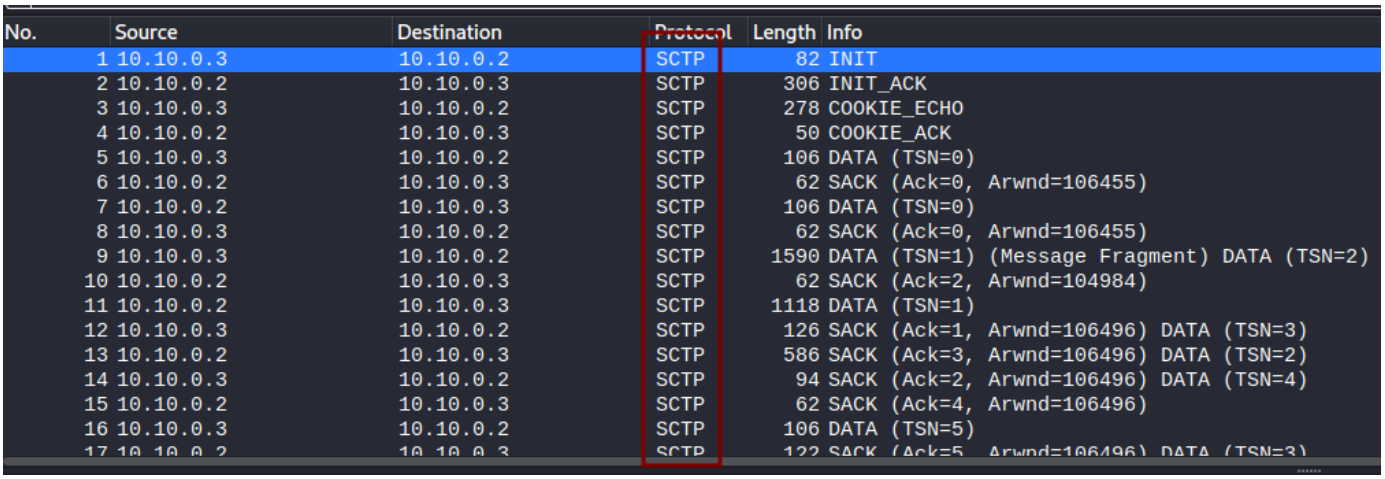


3.3.6. Запускаємо Wireshark на прослуховування бриджа

3.3.7. Підключаємось по SSH до host1 з host2



3.3.8. Перевіряємо, чи дійсно встановлено з’єднання через Wireshark



No.	Source	Destination	Protocol	Length	Info
1	10.10.0.3	10.10.0.2	SCTP	82	INIT
2	10.10.0.2	10.10.0.3	SCTP	306	INIT_ACK
3	10.10.0.3	10.10.0.2	SCTP	278	COOKIE_ECHO
4	10.10.0.2	10.10.0.3	SCTP	50	COOKIE_ACK
5	10.10.0.3	10.10.0.2	SCTP	106	DATA (TSN=0)
6	10.10.0.2	10.10.0.3	SCTP	62	SACK (Ack=0, Arwnd=106455)
7	10.10.0.2	10.10.0.3	SCTP	106	DATA (TSN=0)
8	10.10.0.3	10.10.0.2	SCTP	62	SACK (Ack=0, Arwnd=106455)
9	10.10.0.3	10.10.0.2	SCTP	1590	DATA (TSN=1) (Message Fragment) DATA (TSN=2)
10	10.10.0.2	10.10.0.3	SCTP	62	SACK (Ack=2, Arwnd=104984)
11	10.10.0.2	10.10.0.3	SCTP	1118	DATA (TSN=1)
12	10.10.0.3	10.10.0.2	SCTP	126	SACK (Ack=1, Arwnd=106496) DATA (TSN=3)
13	10.10.0.2	10.10.0.3	SCTP	586	SACK (Ack=3, Arwnd=106496) DATA (TSN=2)
14	10.10.0.3	10.10.0.2	SCTP	94	SACK (Ack=2, Arwnd=106496) DATA (TSN=4)
15	10.10.0.2	10.10.0.3	SCTP	62	SACK (Ack=4, Arwnd=106496)
16	10.10.0.3	10.10.0.2	SCTP	106	DATA (TSN=5)
17	10.10.0.2	10.10.0.3	SCTP	122	SACK (Ack=5, Arwnd=106496) DATA (TSN=3)

3.3.9. Заблокуємо ICMP пакети на host1 з використанням SSH підключення з host2

```
root@host1:~# sysctl net.ipv4.icmp_echo_ignore_all=1
net.ipv4.icmp_echo_ignore_all = 1
root@host1:~# ping -c 3 localhost
PING localhost (127.0.0.1) 56(84) bytes of data.

--- localhost ping statistics ---
3 packets transmitted, 0 received, 100% packet loss,

root@host1:~# exit
logout
Connection to localhost closed.
root@host2:/#
```

Додаткові завдання на лабораторну роботу

Завдання 3.4. Розробити власну утиліту для передачі файлів з використанням SCTP протоколу

Завдання 3.5. Дослідити для яких сервісів можливо використовувати тунелювання через SCTP

Контрольні запитання

1) Що таке SCTP?

2) Які ключові відмінності між SCTP та TCP?

3) Чому SCTP не набув широкого використання?

4) Чому є необхідність прокладання тунелю?

5) Яку роль може відігравати SCTP для задачі конфігурування мережі?

Навчальні матеріали та ресурси

1. Жураковський, Б. Ю. Комп’ютерні мережі. Частина 1. Навчальний посібник [Електронний ресурс] : навчальний посібник для студентів спеціальності 121 «Інженерія програмного забезпечення» та 126 «Інформаційні системи та технології», спеціалізації «Інженерія програмного забезпечення інформаційно управляючих систем» та «Інформаційне забезпечення робототехнічних систем» / Б. Ю. Жураковський, І. О. Зенів ; КПІ ім. Ігоря Сікорського. – Електронні текстові дані (1 файл: 4,3 Мбайт). – Київ : КПІ ім. Ігоря Сікорського, 2020. – 328 с. Режим доступу до ресурсу: https://ela.kpi.ua/handle/123456789/36615

2. Жураковський, Б. Ю. Комп’ютерні мережі. Частина 2. Навчальний посібник [Електронний ресурс] : навчальний посібник для студентів спеціальності 121 «Інженерія програмного забезпечення» та 126 «Інформаційні системи та технології», спеціалізації «Інженерія програмного забезпечення інформаційно управляючих систем» та «Інформаційне забезпечення робототехнічних систем» / Б. Ю. Жураковський, І. О. Зенів ; КПІ ім. Ігоря Сікорського. – Електронні текстові дані (1 файл: 4,73 Мбайт). – Київ : КПІ ім. Ігоря Сікорського, 2020. – 372 с. Режим доступу до ресурсу: https://ela.kpi.ua/handle/123456789/36641

3. Жураковський, Б. Ю. Технології та протоколи мультисервісних мереж. Навчальний посібник для виконання лабораторних робіт [Електронний ресурс] : навч. посіб. для студентів спеціальності 126 «Інформаційні системи та технології» денної форми навчання / Жураковський Б. Ю., Нікітін В. А. ; КПІ ім. Ігоря Сікорського. – Електронні текстові дані (1 файл: 10.61 Мбайт). – Київ : КПІ ім. Ігоря Сікорського, 2023. – 96 с. – Режим доступу до ресурсу: https://ela.kpi.ua/handle/123456789/57345